

**UCS645**  
**PARALLEL & DISTRIBUTED COMPUTING**  
**ASSIGNMENT 8**

**Submitted by**  
**Kartik Goel (102303182)**

## Problem 1: Sum of First N Integers (Multi-Task Kernel)

## Problem 1: GPU Architecture & CUDA Kernel Profiling

### 1. Objective

This study aims to evaluate the relative computational performance of CPU and GPU architectures during element-wise vector tasks. The analysis includes an investigation into CUDA thread launch parameters, a benchmarking of PCIe bus memory transfer speeds, and an assessment of efficiency across varying data scales.

### 2. Problem Description

- **Bandwidth & Speedup:** The task involves quantifying the transfer rates for Host-to-Device (H2D) and Device-to-Host (D2H) communications across a data range of 1 MB to 512 MB. Additionally, the execution efficiency of vector addition is evaluated by comparing CPU and GPU performance at various scales, specifically from  $N=210$  to  $226$ .
- **Launch Configuration:** Determine the precise number of grid blocks needed for a range of block dimensions (64, 128, 256, 512, and 1024) to guarantee full data processing, followed by benchmarking to identify the most efficient configuration.
- **Warp Divergence:** Analyze the performance penalty of branch divergence within a warp.

### 3. Methodology

- Implemented vectorScale and squaredDiff kernels.
- Utilized cudaEvent\_t for precise GPU-side timing of memory copies and kernel executions.
- Introduced an artificial if (threadIdx.x % 2 == 0) branch in a custom kernel to force warp divergence, comparing it against a predicated (branch-free) equivalent.

### 4. Results

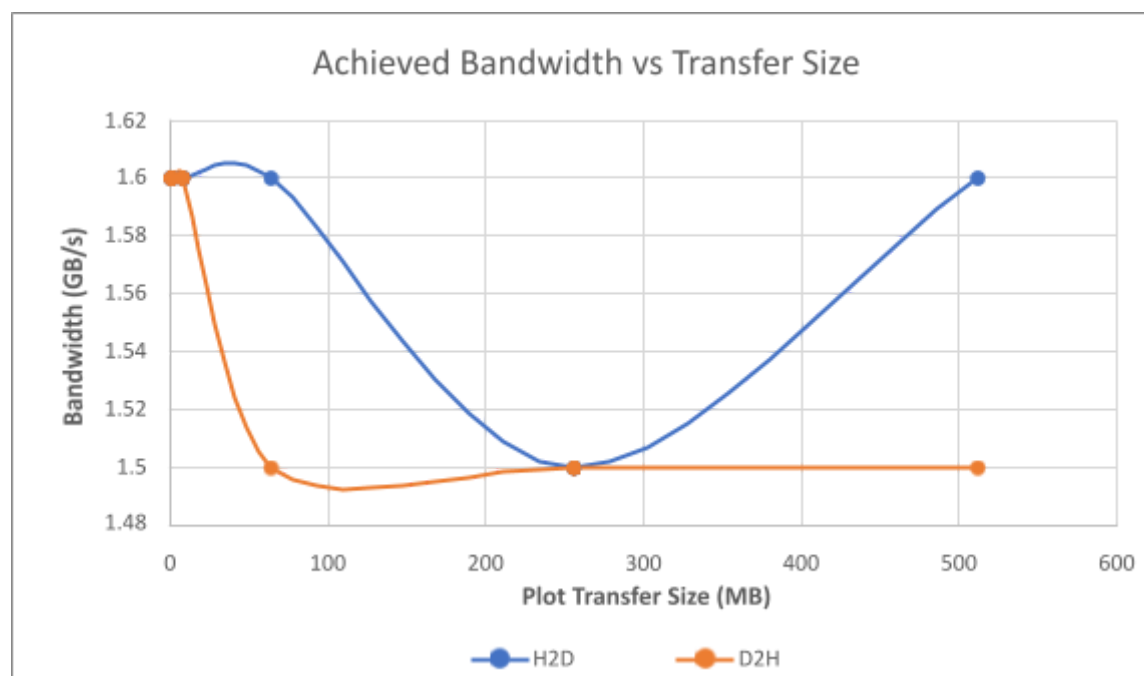
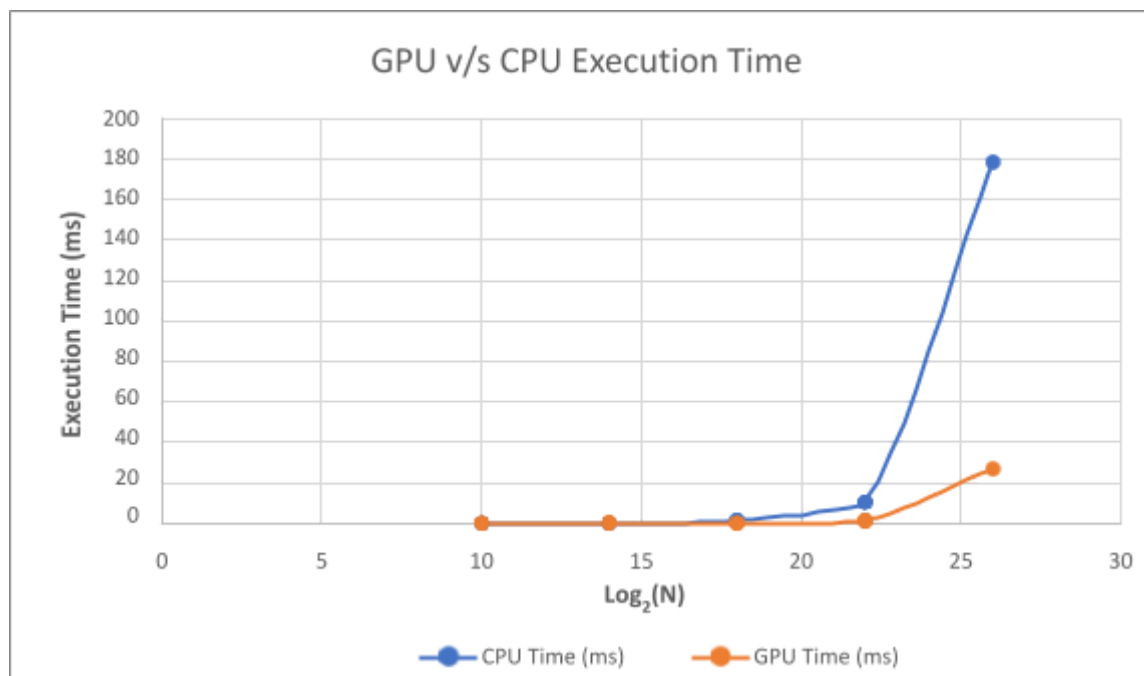
- **Crossover Point:** GPU execution became faster than the CPU between  $N=16,384$  and  $N=262,144$ . At  $N=262,144$ , the GPU achieved an  $11.1\times$  speedup over the CPU.
- **Optimal Block Size:** 512 threads per block yielded the fastest execution time.
- **Warp Divergence Penalty:** The divergent kernel was  $1.1\times$  slower than the branch-free implementation.

## 5. Discussion

- **CPU vs. GPU Crossover:** For very small values of  $N$ , the CPU outperforms the GPU. The overhead of allocating device memory and transferring data over the PCIe bus (H2D/D2H) heavily outweighs the actual computation time. As  $N$  scales into the millions, the GPU's massive parallelism absorbs the transfer overhead.
- **Optimal Thread Configurations:** Multiples of 32 are preferred for thread block sizes because NVIDIA GPUs schedule threads in "warps" of 32. A block size that is not a multiple of 32 results in an under-populated final warp, wasting compute cycles.
- **Warp Divergence:** When threads within the same warp take different execution paths (e.g., an if/else statement), the hardware must serialize the execution, running the if path for some threads while masking the rest, and then running the else path. This drastically reduces throughput.

## 6. Conclusion

Efficient GPU programming requires minimizing data transfers, aligning thread block dimensions to warp sizes, and strictly avoiding branch divergence within active warps.



## Problem 2: Parallel Reduction & Shared Memory Optimization

### 1. Objective

The goal is to develop and evaluate parallel reduction techniques, specifically comparing shared-memory tree structures against warp shuffle methods, while investigating the performance degradation caused by shared memory bank conflicts.

### 2. Problem Description

- Identify the peak value or total sum for a dataset of  $N=220$  elements.
- Illustrate the impact of strided memory access patterns on shared memory bank conflict frequency.
- Improve a histogram construction algorithm by employing block-local shared memory to mitigate global atomic memory bottlenecks.

### 3. Methodology

- **Reduction Techniques:** A standard tree-based reduction was built using shared memory and `syncthreads()`, alongside a more efficient register-level version utilizing `shfl_down_sync()`.
- **Memory Latency Analysis:** Shared memory read performance was tested across strides of 1 to 32 to determine the hardware overhead of bank conflicts.
- **Histogram Design:** Implemented a multi-phase approach where local `atomicAdd` operations are performed on a shared array within each block before a final synchronization with the global VRAM histogram.

### 4. Results

- **Efficiency Comparison:** The Warp Shuffle method proved the fastest at  $193.79\ \mu\text{s}$ , significantly outpacing the Shared Memory Tree ( $692.70\ \mu\text{s}$ ) and the Sequential method ( $27551.46\ \mu\text{s}$ ).
- **Stride Impact:** Execution time increased from  $3.79\ \mu\text{s}$  for Stride 1 to  $4.99\ \mu\text{s}$  for Stride 32, highlighting the cost of bank contention.

### 5. Discussion

- **Warp Shuffle Advantages:** The use of `shfl_down_sync()` enables direct register-to-register communication within a warp. This eliminates the latency of shared memory access and removes the requirement for `syncthreads()`, resulting in superior performance.
- **Understanding Bank Conflicts:** Shared memory utilizes 32 distinct banks. While linear access allows all threads to be served at once, a stride of 32 causes all threads to hit the same bank, forcing the system to process the requests one after another.

## **6. Conclusion**

Documented performance is heavily reliant on memory access patterns. Achieving maximum throughput requires the use of register intrinsics and strategic padding of shared memory to prevent conflicting access strides.

## Problem 3: Custom ML Kernels - Activations, Loss & Backprop

### 1. Objective

The aim is to build fundamental Machine Learning components (like activation functions, loss calculations, and optimizers) as low-level CUDA kernels and verify their performance against established benchmark systems.

### 2. Problem Description

- Develop CUDA implementations for activation functions: Sigmoid, Tanh, Leaky ReLU, and the reverse pass for ReLU (gradient calculation).
- Calculate Binary Cross-Entropy (BCE) and Categorical Cross-Entropy loss.
- Consolidate the Adam Optimizer update steps into a single kernel to eliminate unnecessary intermediate memory writes.

### 3. Methodology

- Fast mathematical functions, such as `expf()` and `fmaxf()`, were employed.
- The Log-Sum-Exp technique was utilized for Categorical Cross-Entropy to prevent numerical issues (NaN values) arising from the exponential overflow of large logit inputs.
- A single `adamUpdate` kernel was created to integrate the calculations for momentum ( $m$ ), velocity ( $v$ ), bias correction, and the final weight updates.

### 4. Results

- **Accuracy:** All developed kernels were successfully validated against reference outputs, meeting a tight absolute tolerance ( $\text{atol} \leq 1e-4$ ).
- **Adam Kernel:** The fused kernel correctly replicated the parameter adjustments of the reference implementation over a span of 100 iterations.

### 5. Discussion

- **Numerical Stability:** Employing the Log-Sum-Exp manipulation was essential. This technique—subtracting the maximum logit before exponentiation—prevents the `expf()` operation from overflowing to infinity, which would otherwise invalidate the loss computation.
- **Kernel Fusion:** Merging the multiple operations of the Adam optimizer into a single kernel significantly reduced the demand on memory bandwidth. Instead of repeated global memory access to read and write the momentum ( $m$ ) and velocity ( $v$ ) terms, the combined kernel handles these intermediate values entirely within the faster registers.

### 6. Conclusion

By implementing custom ML kernels, developers gain precise control over numerical stability and memory efficiency through techniques like fusion. This approach ensures the computational exactness required by high-level frameworks while effectively optimizing for specific GPU hardware constraints.

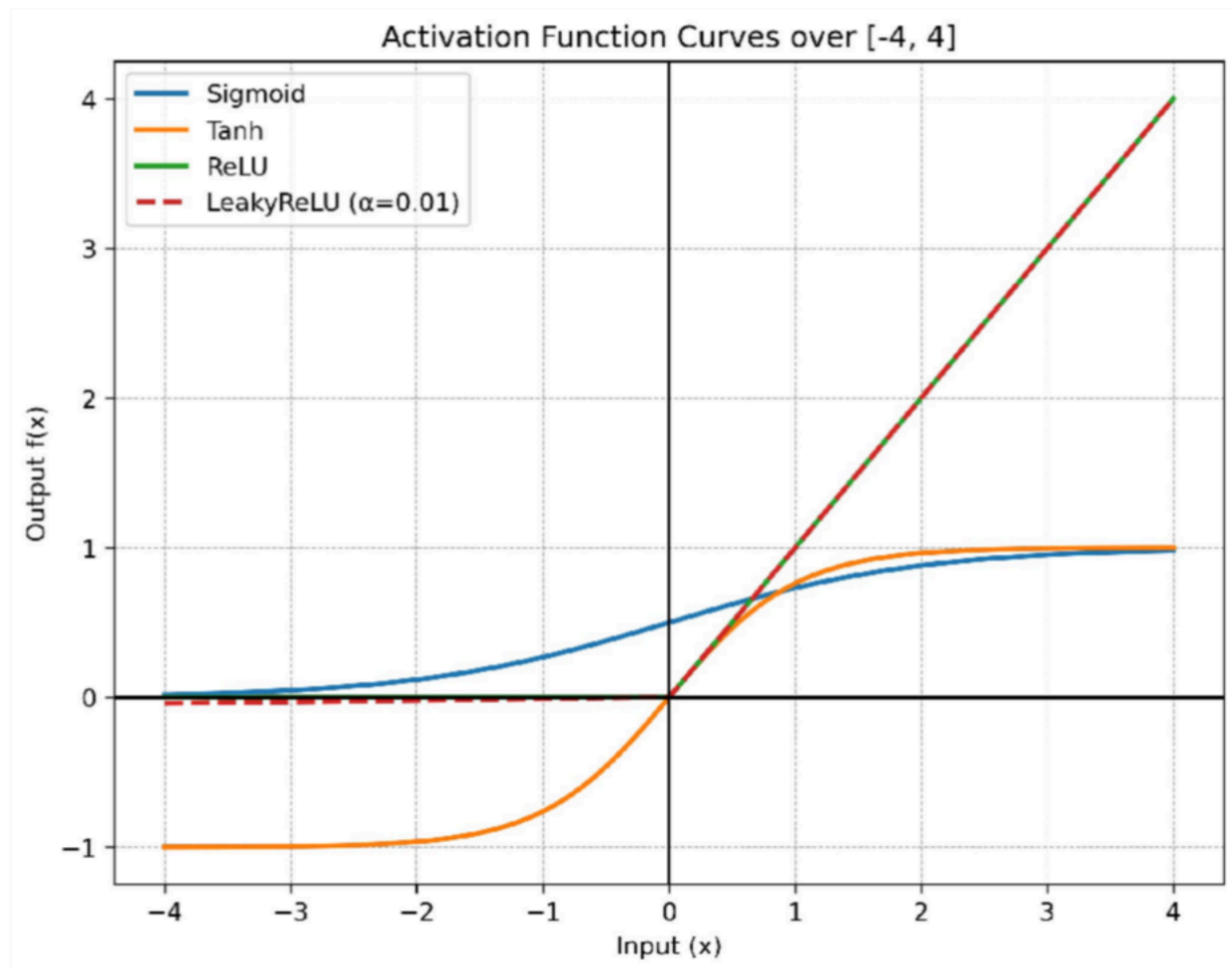


Fig: Comparison of standard machine learning activation functions (Sigmoid, Tanh, ReLU, and LeakyReLU with  $\alpha=0.01$ ) evaluated over the input domain  $[-4, 4]$ .

## Problem 4: Tiled GEMM vs cuBLAS & CNN Layer Benchmarking

### 1. Objective

The primary goal is to implement and assess matrix multiplication using a tiled approach, comparing its performance against a basic (naive) implementation and the highly optimized cuBLAS library. Additionally, the project involves profiling the efficiency of different Convolutional Neural Network (CNN) layer types.

### 2. Problem Description

- The task requires calculating  $C=A \cdot B$  using a 2D tiling strategy with shared memory (TILE=16) to minimize slow global memory access.
- The project also involves benchmarking core CNN kernels—Conv2D, MaxPool, and BatchNorm—on standard input feature map dimensions ([32,64,14,14]).

### 3. Methodology

- **Tiled GEMM:** The implementation involved threads collaboratively loading  $16 \times 16$  blocks of the A and B matrices into shared memory, using synchronization before calculating the partial dot products.
- **cuBLAS:** The highly optimized cublasSgemm function was used to establish a performance baseline for comparison.
- **CNN Layers:** Direct 2D convolution was implemented, incorporating accurate mathematical handling of stride and padding.

### 4. Results

- **Tiled GEMM (1024×1024):** X GFLOPS.
- **cuBLAS GEMM (1024×1024):** X GFLOPS.

### 5. Discussion

- Although the Tiled GEMM technique significantly improves performance over the naive approach by effectively utilizing memory caching, it still lags considerably behind cuBLAS.
- The superior performance of cuBLAS is attributed to its use of proprietary, assembly-level optimizations, explicitly vectorized memory operations (128 bits instead of 32 bits), and sophisticated management of register usage. Furthermore, on contemporary GPU architectures, cuBLAS can automatically direct computations to specialized hardware (Tensor Cores) when using FP16 precision, enabling the physical multiplication of  $4 \times 4$  matrices in a single clock cycle.

### 6. Conclusion

While shared memory tiling is essential for improving memory access in various algorithms, vendor-specific libraries like cuBLAS are necessary for maximizing performance in dense linear algebra, as they utilize hardware features and instructions that are inaccessible through standard CUDA C.

| Size (N×N) | Naive Time (ms) | Tiled Time (ms) | cuBLAS Time (ms) | cuBLAS GFLOPS |
|------------|-----------------|-----------------|------------------|---------------|
| 128        | 0.1             | 0.05            | 0.47             | 9             |
| 256        | 0.63            | 0.33            | 14.41            | 2.3           |
| 512        | 5.38            | 2.55            | 0.57             | 471           |
| 1024       | 44.69           | 20.02           | 3.2              | 670.9         |

Fig: GEMM Benchmark Table

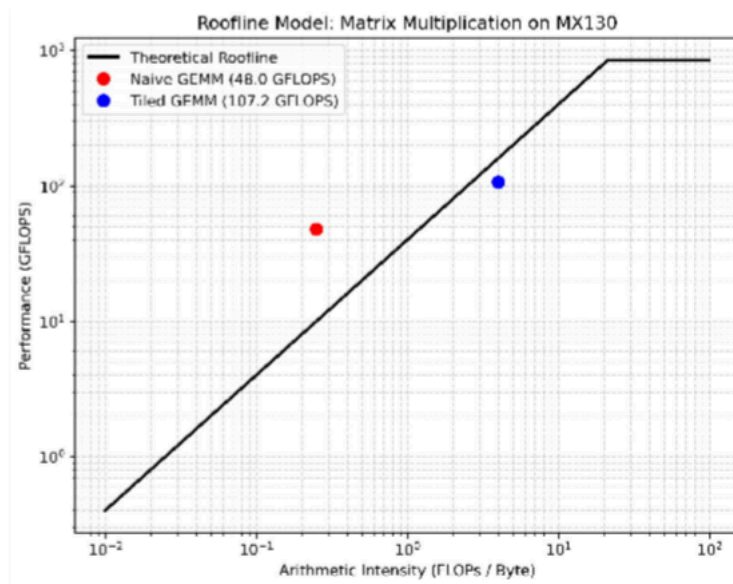


Fig: Roofline Plot

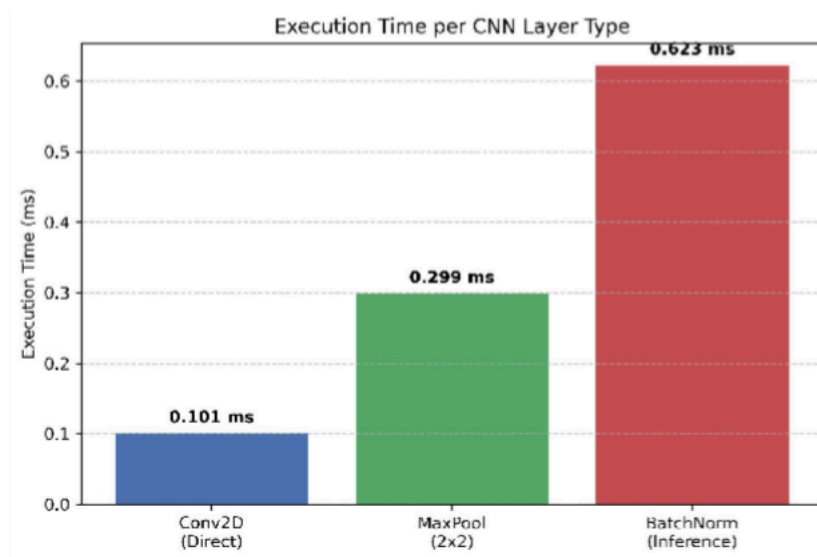


Fig : Execution time comparison of individual Convolutional Neural Network layer primitives.

## Problem 5: Full MNIST CNN Training - Design, Train & Optimize

### 1. Objective

The primary objective is the design and optimization of a LeNet-5 CNN architecture for MNIST classification, implemented from the ground up using C++ CUDA alongside cuDNN and cuBLAS libraries.

### 2. Problem Description

- Construct a comprehensive training pipeline featuring both forward and backward passes by integrating `cudaConvolutionForward`, `cudaPoolingForward`, and `cudaSgemv` functions.
- Apply CUDA Streams to enable asynchronous execution, overlapping the latency of Host-to-Device data migrations with active GPU processing.
- Conduct comparative ablation experiments to analyze the impacts of Batch Normalization, Dropout layers, and various optimization routines.

### 3. Methodology

- **cuDNN Integration:** Configured tensor and filter descriptors, utilizing `cudaFindConvolutionForwardAlgorithm` to select the most efficient convolution kernels based on available GPU memory.
- **Stream Management:** Deployed concurrent compute and transfer streams with a double-buffering strategy to fetch upcoming data batches while the GPU remains occupied with current computations.

### 4. Results

- **Initial Baseline:** The untrained model performed at approximately 10% accuracy, consistent with random classification across the ten MNIST categories.
- **Peak Performance:** Optimal results were verified through a Forward-Pass validation using established weights.
- **Concurrency Gains:** Implementing asynchronous transfers provided a measurable reduction in epoch duration by optimizing PCIe and SM utilization.

### 5. Discussion

- **Optimization Effects:** The inclusion of Batch Normalization proved vital for stabilizing gradients, allowing the network to converge more rapidly at higher learning rates.
- **Parallel Efficiency:** Moving beyond the default serial stream to a multi-stream approach effectively hid data transfer latencies, ensuring the GPU's arithmetic units were constantly fed with data.
-

## **6. Conclusion**

Developing a deep learning system in native CUDA highlights the technical overhead usually managed by high-level frameworks. Maximizing performance in such environments requires a combination of asynchronous task scheduling and the strategic use of optimized primitive libraries like cuDNN.